



UNIVERSIDAD TÉCNICA PARTICULAR DE LOJA
La Universidad Católica de Loja

CARRERA DE COMPUTACIÓN

Integrantes: Ingrid Micaela Pardo Gualán
María Emilia Solano Ramírez

Programación Orientada a Objetos

Bimestre: Segundo Bimestre

Profesor: Wayner Bustamante.

Fecha de entrega:

24 de julio del 2026

1. Aplicación de Principios SOLID

1.1 Principio de Responsabilidad Única (SRP)

Cada clase tiene una única razón para cambiar: `ConsoleView` gestiona exclusivamente la interacción por consola; los controladores (`ActivoController`, `MantenimientoController`) solo orquestan llamadas a la capa de servicio; los servicios (`ActivoServiceImpl`, `MantenimientoServiceImpl`) concentran las reglas de negocio y validaciones; y los repositorios (`ActivoRepositorySQLite`, `MantenimientoRepositorySQLite`) se ocupan únicamente de la persistencia en SQLite.

1.2 Principio de Abierto/Cerrado (OCP)

La clase `Activo` está diseñada para permitir la incorporación de nuevos tipos de activos sin modificar el código existente. Cada subtipo (`Hardware`, `Periférico`, `Licencia`) implementa su propia versión de `calcularCostoMantenimiento()` y `obtenerTipo()`; un nuevo tipo de activo solo requeriría extender `Activo` e implementar estos dos métodos, sin alterar las clases de servicio ni de controlador. Esta intención queda explícita en el propio código fuente, en el comentario asociado al método `obtenerTipo()`.

1.3 Principio de Sustitución de Liskov (LSP)

Las subclases `Hardware`, `Periferico` y `Licencia` pueden utilizarse en cualquier contexto donde se espere una referencia de tipo `Activo` sin alterar la corrección del programa. Esto se evidencia en el uso de `List<Activo>` en `IActivoService` y `ActivoRepository`, y en el método `toString()` de `Activo`, que invoca de forma polimórfica `calcularCostoMantenimiento()` y `obtenerTipo()` sobre cualquier subtipo concreto.

1.4 Principio de Segregación de Interfaces (ISP)

En lugar de definir una única interfaz genérica de persistencia o de servicio, el sistema separa las responsabilidades en interfaces específicas: `ActivoRepository` y `MantenimientoRepository` para persistencia, e `IActivoService` e `IMantenimientoService` para lógica de negocio. Ningún cliente se ve obligado a depender de operaciones que no utiliza.

1.5. Principio de Inversión de Dependencias (DIP)

2.2. Explicación del UML

El sistema está organizado en una arquitectura por capas que separa la interacción con el usuario, la lógica de negocio, el acceso a datos y el modelo de dominio. Esta organización favorece el bajo acoplamiento entre componentes y facilita el mantenimiento y la extensión del sistema.

2.2.1. ConsoleVista

Clase de la capa de presentación. Contiene un Scanner para leer datos del usuario y referencia **ActivoController** y **MantenimientoController**.

Relación: Asociación (línea con flecha simple) hacia **ActivoController** y **MantenimientoController**. Los usa para ejecutar cada opción del menú (registrar, listar, buscar, asignar, eliminar activos; registrar mantenimiento, ver historial, ver costo total).

No se relaciona directamente con la capa de servicio ni de persistencia.

2.2.2. ActivoController

Controlador que expone operaciones sobre activos a la vista.

Relación: Asociación con **IActivoService** (depende de la interfaz, no de la implementación). Delega en ella los métodos registrar, actualizar, listar, buscar, asignar, eliminar.

Es utilizado por **ConsoleVista**.

2.2.3. MantenimientoController (MantenimientoController)

Controlador que expone operaciones sobre mantenimientos.

Relación: Asociación con **IMantenimientoService**. Delega registrarMantenimiento, historialDe, costoTotal.

Es utilizado por ConsoleVista.

2.2.4. IActivoService (interfaz)

Contrato de negocio para activos: registrarActivo, actualizarActivo, listarActivos, buscarActivo, asignarActivo, eliminarActivo.

Relación: Es implementada (realización, flecha punteada con triángulo hueco) por `ActivoServiceImpl`.

Es usada (dependencia) por **ActivoController**.

2.2.5. `ActivoServiceImpl`

Implementación de **`IActivoService`**. Contiene la lógica de negocio y validaciones (`validarActivo`).

Relaciones:

Implementa **`IActivoService`**.

Asociación con **`ActivoRepository`** (atributo `repository`) para persistir y consultar activos.

Dependencia con **`ActivoNoEncontradoException`**, que lanza cuando un activo no existe.

2.2.6. **`IMantenimientoService`** (interfaz)

Contrato de negocio para mantenimientos: `registrarMantenimiento`, `historialDe`, `costoTotalMantenimiento`.

Relación: Implementada por **`MantenimientoServiceImpl`**. Usada por **`MantenimientoController`**.

2.2.7. `MantenimientoServiceImpl`

Implementación de **`IMantenimientoService`**.

Relaciones:

Implementa **`IMantenimientoService`**.

Asociación con **`MantenimientoRepository`** (para guardar/consultar registros de mantenimiento) y con **`ActivoRepository`** (para buscar el activo y actualizar su estado a `EN_MANTENIMIENTO`).

Dependencia con **`ActivoNoEncontradoException`**.

2.2.8. **`ActivoRepository`** (interfaz)

Contrato de persistencia para activos: `guardar`, `actualizar`, `eliminar`, `buscarPorCodigo`, `listarTodos`, `existe`.

Relación: Implementada por **ActivoRepositorySQLite**. Usada por **ActivoServiceImpl** y **MantenimientoServiceImpl**.

2.2.9. ActivoRepositorySQLite

Implementación concreta de **ActivoRepository** sobre SQLite.

Relaciones:

Implementa **ActivoRepository**.

Asociación con **ActivoFactory** (atributo factory) para reconstruir objetos Activo desde un ResultSet.

Dependencia con **ConexionSQLite** (obtiene la conexión JDBC en cada operación).

Dependencia con **ActivoNoEncontradoException** (uso indirecto vía la excepción de persistencia).

2.2.10. MantenimientoRepository (interfaz)

Contrato de persistencia para mantenimientos: guardar, listarPorAActivo, costoTotal.

Relación: Implementada por **MantenimientoRepositorySQLite**. Usada por **MantenimientoServiceImpl**.

2.2.11. MantenimientoRepositorySQLite

Implementación concreta sobre SQLite.

Relaciones:

Implementa **MantenimientoRepository**.

Dependencia con **ConexionSQLite** para obtener la conexión.

Crea la tabla mantenimientos con clave foránea hacia la tabla activos.

2.2.12. ActivoFactory (interfaz)

Define crearDesdeResultado(ResultSet): Activo.

Relación: Implementada por **ActivoFactoryImpl**. Usada (asociación) por **ActivoRepositorySQLite**.

2.2.13. ActivoFactoryImpl

Implementación de ActivoFactory.

Relación: Crea instancias de Hardware, Periferico o Licencia según el campo tipo leído del ResultSet (dependencia hacia las tres subclases de Activo).

2.2.14. ConexionSQLite

Clase utilitaria (Singleton) con métodos estáticos obtenerConexion() y cerrarConexion().

Relación: Es dependencia (no asociación permanente) de **ActivoRepositorySQLite** y **MantenimientoRepositorySQLite**, que la invocan para obtener la conexión JDBC.

2.2.15. ActivoNoEncontradoException

Excepción de negocio.

Relación: Dependencia desde **ActivoServiceImpl**, **MantenimientoServiceImpl** y **ActivoRepositorySQLite**, quienes la lanzan cuando no existe un activo con el código solicitado.

2.2.16. Activo (clase abstracta)

Superclase del dominio: atributos codigo, nombre, fechaAdquisicion, estado; métodos abstractos calcularCostoMantenimiento() y obtenerTipo().

Relaciones:

Herencia (triángulo hueco) hacia **Hardware**, **Periferico** y **Licencia**.

Asociación con EstadoActivo (atributo estado).

2.2.17. Hardware

Subclase de Activo. Atributos propios: ramGb, procesador.

Relación: Herencia de Activo. Es creada por **ActivoFactoryImpl**. Es usada en las pruebas de **Activo Test**.

2.2.18. Periferico

Subclase de Activo. Atributo propio: **tipoPeriferico**.

Relación: Herencia de Activo. Creada por **ActivoFactoryImpl**. Usada por Activo Test.

2.2.19. Licencia

Subclase de Activo. Atributo propio: fechaExpiracion.

Relación: Herencia de Activo. Creada por **ActivoFactoryImpl**. Usada por Activo Test.

2.2.20. EstadoActivo (enumeración)

Valores: DISPONIBLE, ASIGNADO, EN_MANTENIMIENTO, DE_BAJA.

Relación: Asociación con Activo (representa su estado).

2.2.21. RegistroMantenimiento

Clase independiente: id, codigoActivo, fecha, costo.

Relación: Asociación con **MantenimientoRepository/MantenimientoRepositorySQLite** (se guarda y consulta a través de ellos) y con **MantenimientoServiceImpl** (la crea y retorna).

2.2.22. Main

Clase con el método estático main(String[]).

Relación: Dependencia hacia **ActivoRepositorySQLite, MantenimientoRepositorySQLite, ActivoServiceImpl, MantenimientoServiceImpl, ActivoController, MantenimientoController y ConsoleVista**. Es el único punto donde se instancian clases concretas y se inyectan por constructor (composition root).

2.2.21. Activo Test (<<JUnit Test>>)

Clase de pruebas unitarias.

Relación: Dependencia hacia Hardware, Periferico y Licencia, a las que instancia para verificar calcularCostoMantenimiento() y obtenerTipo().

3. Explicación De Los Patrones Aplicados Y Fragmentos Relevantes

3.1 Arquitectura En Capas (Vista – Controlador – Servicio – Repositorio)

El sistema separa la presentación, la coordinación de solicitudes, la lógica de negocio y el acceso a datos en paquetes independientes (vista, controlador, servicio, repositorio), reduciendo el acoplamiento entre la interfaz de usuario y la persistencia.

3.2 Repository (Repositorio)

Las interfaces ActivoRepository y MantenimientoRepository abstraen el mecanismo de almacenamiento; las clases de servicio operan sobre estas interfaces sin conocer que la implementación actual utiliza SQLite. Esto permitiría sustituir el motor de persistencia sin modificar la capa de negocio.

public interface ActivoRepository {

`void guardar(Activo activo);`

`void actualizar(Activo activo);`

`void eliminar(String codigo);`

`Activo buscarPorCodigo(String codigo);`

`List<Activo> listarTodos();`

`boolean existe(String codigo);`

}

3.3 Factory Method

La interfaz ActivoFactory y su implementación ActivoFactoryImpl encapsulan la creación del subtipo concreto de Activo a partir del campo "tipo" leído desde el ResultSet, evitando que ActivoRepositorySQLite conozca los detalles de instanciación de cada subtipo:

switch (tipo) {

`case "HARDWARE":`

```

        return new Hardware(codigo, nombre, fechaAdquisicion, estado,

            rs.getInt("ram_gb"), rs.getString("procesador"));

    case "PERIFERICO":

        return new Periferico(codigo, nombre, fechaAdquisicion, estado,

            rs.getString("tipo_periferico"));

    case "LICENCIA":

        return new Licencia(codigo, nombre, fechaAdquisicion, estado, fechaExpiracion);

    default:

        throw new SQLException("Tipo de activo desconocido: " + tipo);

}

```

3.4 Singleton

La clase ConexionSQLite implementa el patrón Singleton mediante un constructor privado, un atributo estático de conexión y un método estático de acceso que crea la conexión únicamente si no existe o está cerrada:

```

private ConexionSQLite() { }

public static Connection obtenerConexion() throws SQLException {

    if (conexion == null || conexion.isClosed()) {

        Class.forName("org.sqlite.JDBC");

        conexion = DriverManager.getConnection(URL);

    }

    return conexion;

}

```

3.5 Inyección De Dependencias Por Constructor

Los controladores y servicios reciben sus dependencias (interfaces) a través del constructor, y es la clase Main quien ensambla el grafo de objetos concreto de la aplicación:

```
ActivoRepository activoRepository = new ActivoRepositorySQLite();
```

```
IActivoService activoService = new ActivoServiceImpl(activoRepository);
```

```
ActivoController activoController = new ActivoController(activoService);
```

```
ConsoleView vista = new ConsoleView(activoController, mantenimientoController);
```

3.6 Excepción De Negocio Personalizada

ActivoNoEncontradoException extiende RuntimeException para representar de forma explícita, dentro del dominio del problema, la ausencia de un activo con un código determinado, en lugar de propagar excepciones genéricas o nulas:

```
public class ActivoNoEncontradoException extends RuntimeException {  
  
    public ActivoNoEncontradoException(String codigo) {  
  
        super("No existe un activo con codigo: " + codigo);  
  
    }  
  
}
```

3.7 Pruebas Unitarias

La clase ActivoTest (JUnit) valida el cálculo polimórfico de calcularCostoMantenimiento() y obtenerTipo() para los tres subtipos de Activo (Hardware, Periferico, Licencia), verificando de forma automatizada las reglas de negocio asociadas a cada tipo de activo.